

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

data = pd.read_csv('knn_handwritten_digits_500.csv')

print(data.shape)
print(data.head())

```

(500, 65)

	pixel_1	pixel_2	pixel_3	pixel_4	pixel_5	pixel_6	pixel_7
0	0.0	0.0	0.0	7.0	12.0	0.0	0.0
1	0.0	0.0	11.0	16.0	8.0	0.0	0.0
2	0.0	0.0	8.0	15.0	12.0	4.0	0.0
3	0.0	0.0	2.0	12.0	12.0	12.0	9.0
4	0.0	2.0	13.0	16.0	10.0	0.0	0.0

pixel_8 \

	pixel_9	pixel_10	...	pixel_56	pixel_57	pixel_58	pixel_59
0	0.0	0.0	...	2.0	0.0	0.0	0.0
1	0.0	6.0	...	0.0	0.0	0.0	13.0
2	0.0	5.0	...	0.0	0.0	0.0	13.0
3	0.0	0.0	...	0.0	0.0	0.0	3.0
4	0.0	6.0	...	0.0	0.0	3.0	15.0

pixel_60 \

	pixel_61	pixel_62	pixel_63	pixel_64	label
0	14.0	14.0	5.0	0.0	6
1	11.0	1.0	0.0	0.0	9
2	15.0	8.0	0.0	0.0	3
3	3.0	0.0	0.0	0.0	7
4	16.0	10.0	1.0	0.0	2

[5 rows x 65 columns]

```

data.isnull().sum()

pixel_1      0
pixel_2      0
pixel_3      0
pixel_4      0
pixel_5      0
..
pixel_61     0
pixel_62     0
pixel_63     0
pixel_64     0
label        0
Length: 65, dtype: int64

print(data['label'].value_counts().sort_index())

label
0      48
1      45
2      44
3      53
4      56
5      64
6      49
7      47
8      40
9      54
Name: count, dtype: int64

x = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=5,
    stratify=y
)

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

Training data: (400, 64)
Testing data: (100, 64)

sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

```

```

model = KNeighborsClassifier(
    n_neighbors=3,
    metric='minkowski',
    p=2
)

model.fit(x_train, y_train)

KNeighborsClassifier(n_neighbors=3)

y_pred = model.predict(x_test)

print("Predicted values:")
print(y_pred)

print("\nActual values:")
print(y_test)

Predicted values:
[5 2 4 9 3 3 6 3 7 0 5 9 4 1 0 5 7 6 9 9 6 8 2 1 8 4 3 0 5 3 3 5 8 7 5
 3 9
 2 0 3 4 1 4 3 6 8 6 1 9 7 4 9 4 9 1 4 2 2 0 1 2 5 5 3 3 7 3 1 8 6 1 7
 2 6
 0 6 2 5 2 9 5 1 1 7 4 8 0 5 8 7 7 4 4 0 3 6 6 5 0 5]

Actual values:
[5 2 4 9 3 3 6 3 7 0 5 9 4 1 0 5 7 6 9 9 6 8 2 1 8 4 9 0 5 3 3 5 8 7 5
 3 9
 2 0 3 4 8 4 9 6 8 6 1 9 7 4 9 4 9 1 4 2 2 0 1 2 5 5 3 3 7 3 1 8 6 1 7
 2 6
 0 6 2 5 2 9 5 1 1 7 4 8 0 5 8 7 7 4 4 0 3 6 6 5 0 5]

accuracy_score = metrics.accuracy_score(
    y_test,
    y_pred
)

print("Accuracy Score:", accuracy_score)

print(
    "Accuracy in Percentage:",
    int(accuracy_score * 100),
    "%"
)

Accuracy Score: 0.97
Accuracy in Percentage: 97 %

print("Classification Report:")
print(
    metrics.classification_report(

```

```

        y_test,
        y_pred
    )
)

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9
1	0.90	1.00	0.95	9
2	1.00	1.00	1.00	9
3	0.85	1.00	0.92	11
4	1.00	1.00	1.00	11
5	1.00	1.00	1.00	13
6	1.00	1.00	1.00	10
7	1.00	1.00	1.00	9
8	1.00	0.88	0.93	8
9	1.00	0.82	0.90	11
accuracy			0.97	100
macro avg	0.97	0.97	0.97	100
weighted avg	0.97	0.97	0.97	100

```

conf_mat = metrics.confusion_matrix(
    y_test,
    y_pred
)

print("Confusion Matrix:")
print(conf_mat)

Confusion Matrix:
[[ 9  0  0  0  0  0  0  0  0  0]
 [ 0  9  0  0  0  0  0  0  0  0]
 [ 0  0  9  0  0  0  0  0  0  0]
 [ 0  0  0 11  0  0  0  0  0  0]
 [ 0  0  0  0 11  0  0  0  0  0]
 [ 0  0  0  0  0 13  0  0  0  0]
 [ 0  0  0  0  0  0 10  0  0  0]
 [ 0  0  0  0  0  0  0  9  0  0]
 [ 0  1  0  0  0  0  0  0  7  0]
 [ 0  0  0  2  0  0  0  0  0  9]]

plt.figure(figsize=(8, 6))

sn.heatmap(
    conf_mat,
    annot=True,
    fmt='d'
)

```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('KNN - Handwritten Digit Recognition')
plt.show()
```

